UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación

Quinta Práctica Calificada

CC112 **02/12/2025 Tiempo: 2:30 horas** **Ciclo: 2025-II**

Normas:

1. No compartir respuestas/consultas con sus compañeros a través de chats, redes sociales u otros medios digitales. No se permiten apuntes de clase.
2. Todo acto anti-ético será amonestado y registrado en el historial del estudiante.
3. No se permite presentar archivos .exe o encabezados como `<bits/stdc++.h>`, `<vector>`, `<algorithm>` u otras que no hayan sido vistas o autorizadas en clase.

---

1. [5 puntos] Un qubit se puede escribir como una combinación lineal de los estados base $|0\rangle$ y $|1\rangle$:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle,$$

donde α y β son números reales y se cumple: $\alpha^2 + \beta^2 = 1$.
Cuando tenemos **más de un qubit**, el estado del sistema completo se obtiene mediante el **producto tensorial** de los estados individuales.

Ejemplo de producto tensorial (3 qubits):

$$|q_1\rangle = a_0\,|0\rangle + a_1\,|1\rangle$$
$$|q_2\rangle = b_0\,|0\rangle + b_1\,|1\rangle$$
$$|q_3\rangle = c_0\,|0\rangle + c_1\,|1\rangle$$

El estado total de los **3 qubits** se obtiene con el producto tensorial:

$$|q_1q_2q_3\rangle = |q_1\rangle \otimes |q_2\rangle \otimes |q_3\rangle$$

Primero hacemos $|q_1\rangle \otimes |q_2\rangle$:

$$|q_1q_2\rangle = a_0b_0\,|00\rangle + a_0b_1\,|01\rangle + a_1b_0\,|10\rangle + a_1b_1\,|11\rangle$$

Ahora tomamos ese resultado y lo tensoreamos con $|q_3\rangle$:

$$|q_1q_2q_3\rangle = (a_0b_0\,|00\rangle + a_0b_1\,|01\rangle + a_1b_0\,|10\rangle + a_1b_1\,|11\rangle) \otimes (c_0\,|0\rangle + c_1\,|1\rangle)$$

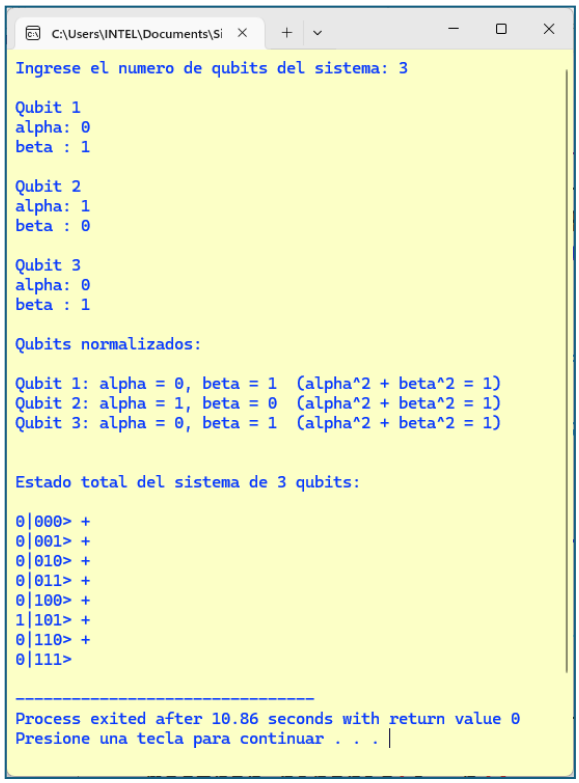
Distribuimos:

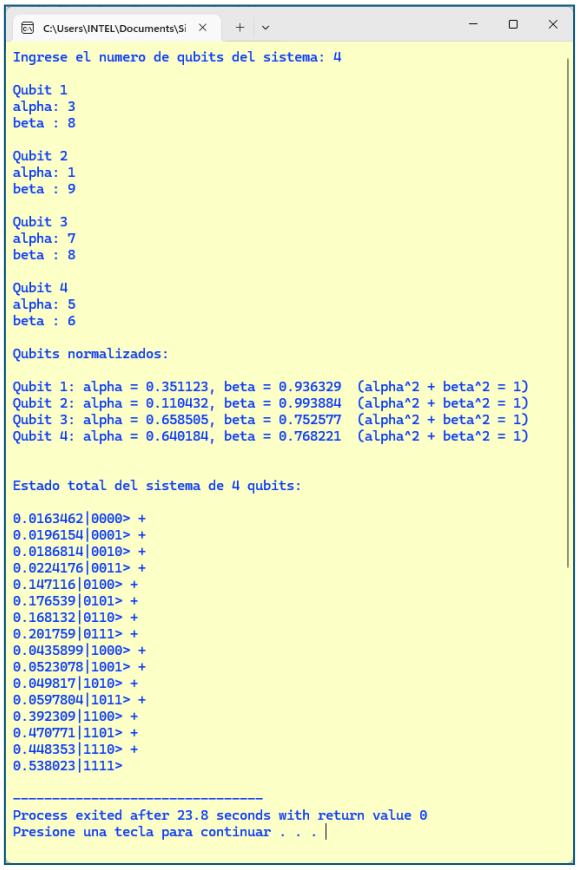
$$\begin{aligned}|q_1q_2q_3\rangle = \quad & a_0b_0c_0\,|000\rangle + a_0b_0c_1\,|001\rangle + \\ & a_0b_1c_0\,|010\rangle + a_0b_1c_1\,|011\rangle + \\ & a_1b_0c_0\,|100\rangle + a_1b_0c_1\,|101\rangle + \\ & a_1b_1c_0\,|110\rangle + a_1b_1c_1\,|111\rangle\end{aligned}$$

Implementar una clase Qubit que:

1. Pida al usuario cuántos qubits tiene el sistema.
2. Lea las amplitudes α y β de cada qubit, normalizándolas.
3. Calcule el **producto tensorial** de los n qubits.
4. Muestre todas las amplitudes del estado total, indicando a qué estado base $|\, b_{n-1} \ldots b_0\rangle$ corresponde cada una.

`Ejemplo de ejecución:`





2. [5 puntos] Desarrollar un programa para préstamo de libros en una biblioteca utilizando Programación Orientada a Objetos en C++. El sistema debe trabajar con una clase **Libro**, que representa cada ejemplar disponible para préstamo.

Cree la clase **Libro** con los siguientes **atributos privados**:
titulo (cadena): nombre del libro, **autor** (cadena): autor del libro, **código** (entero): identificador único del libro, **prestado** (booleano): indica si el libro está prestado o disponible, **vecesPrestado** (entero): cantidad de veces que el libro ha sido prestado.
Implemente los siguientes **métodos públicos**:

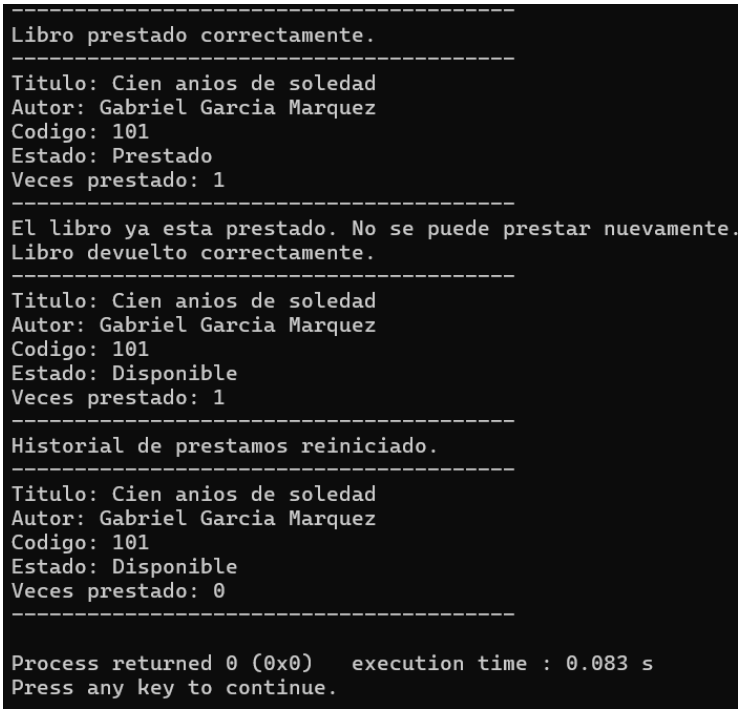
1. **prestar()**
   - Marca el libro como prestado si actualmente está disponible.
   - Incrementa en 1 el contador *vecesPrestado*.
   - Si ya está prestado, el método debe mostrar un mensaje indicando que no puede prestarse.

2. **devolver()**
   - Marca el libro como disponible si actualmente está prestado.
   - Si no estaba prestado, mostrar un mensaje indicando que no había préstamo activo.
3. **mostrarInformacion()**
   - Muestra en pantalla: título, autor, código, estado (prestado/disponible) y número de veces prestado.
4. **resetearHistorial()**
   - Reinicia a 0 el número de veces prestado (por ejemplo, al final del semestre).

Restricciones y reglas del sistema

- No se permite prestar un libro que ya está prestado.
- No se permite devolver un libro que no estaba prestado.
- El código del libro debe ser un número entero positivo.
- El sistema debe validar que el título y el autor no sean cadenas vacías.

Ejemplo de Salida:



3. [5 puntos] Un banco necesita ofrecer una cuenta limitada a 5 transacciones diarias. Escriba una clase **CuentaLimitada** que gestione las transacciones, debe tener como miembros: int numeroCuenta, char* titular, float saldo, Transaccion***** historial, int numTransacciones, int capacidadHistorial. La estructura **Transaccion** cuenta con campos: char tipo, float monto. El tipo es un carácter 'D' o 'R' para deposito o retiro respectivamente. Escriba los métodos de constructor y destructor de forma adecuada. Tenga en cuenta que solo los depósitos o retiros exitosos cuentan como transacciones. La clase también implementa los métodos void mostrarInformacion() const, void mostrarHistorial() const; para mostrar información de la cuenta y el listado de transacciones respectivamente.

Ejemplo: Se crea una cuenta para Juan Perez y luego realiza las siguientes transacciones, depósito de 200, retiro de 300, depósito de 300, retiro de 100, retiro de 200, depósito de 300.

```
 Deposito exitoso: +200 | Saldo actual: 200
Error: Fondos insuficientes. Saldo: 200, Intento: 300
 Deposito exitoso: +300 | Saldo actual: 500
   Retiro exitoso: -100 | Saldo actual: 400
   Retiro exitoso: -200 | Saldo actual: 200
 Deposito exitoso: +300 | Saldo actual: 500
Alcanzo el maximo numero de transacciones

=== INFORMACION DE CUENTA ===
Numero de cuenta: 1
Titular: Juan Perez
Saldo actual: $500
Transacciones registradas: 5
============================


=== HISTORIAL DE TRANSACCIONES ===
Cuenta: 1 - Juan Perez
--------------------------------------------------
      Tipo     Monto
--------------------------------------------------
  Deposito       200
  Deposito       300
    Retiro       100
    Retiro       200
  Deposito       300
--------------------------------------------------
Saldo final: $500
```

4. [5 puntos]

a) Implemente una clase llamada **`Contador`** que cumpla con las siguientes especificaciones:

i) Debe tener un atributo entero privado llamado **`valor`**.

ii) Debe incluir:

- Un **constructor por defecto** que inicialice `valor` en 0.
- Un **constructor con parámetro** que reciba un entero y lo asigne a `valor`.
- Un **destructor** que muestre un mensaje indicando que el objeto está siendo destruido.

iii) Debe implementar un método: `Contador& incrementar(int cantidad);`
El método debe sumar `cantidad` al atributo `valor` y **retornar el propio objeto usando el puntero `this`**, permitiendo encadenar llamadas.

iv) Debe implementar un método: `void mostrar() const;` que imprima el valor actual del contador.

v) La siguiente instrucción debe ejecutarse correctamente con su implementación:
`c.incrementar(5).incrementar(10).mostrar();`

b) Implementar una clase `X`, para que la siguiente instrucción sea válida

```
v1()(7)()[4] = 100;
```

donde `v1` es un objeto de la clase `X`.